

Optimal Cell Resizing in OpenROAD-flow-scripts Using Optimization Modulo Theories

Chad Baker, Raphael Huang, Wajid Ali, Anil Muddamsetti
Dept. of Computer Science and Engineering
University of California, Santa Cruz
Santa Cruz, USA

Abstract—Timing closure is a critical step in physical design, ensuring data propagates through a circuit within a clock cycle. Traditional EDA tools rely on heuristic-based cell resizing to fix timing violations, which often settle for local optima. This project explores the use of Optimization Modulo Theories (OMT), specifically the Z3 solver, to perform formal, global cell resizing on critical data paths extracted from the OpenROAD flow. By modeling the timing behavior of the SkyWater 130nm library using non-linear arithmetic constraints, we constructed a closed-loop optimization pipeline. Our results demonstrate that formal optimization can outperform standard heuristics, achieving a 0.02 ns slack improvement on the GCD benchmark by leveraging global path context to make counter-intuitive sizing decisions.

Index Terms—Formal Methods, VLSI, SMT, OMT, OpenROAD, Timing Optimization

I. INTRODUCTION

Modern semiconductor physical design transforms a synthesized logical netlist into a manufacturable layout. A central challenge in this process is *timing closure*: ensuring that signals arrive at storage elements (flip-flops) within specified setup and hold windows. As technology nodes shrink, the complexity of delay optimization increases, leading Electronic Design Automation (EDA) tools to rely heavily on greedy heuristics for gate sizing. While computationally efficient, these heuristics often get trapped in local optima, leaving potential performance gains on the table.

This project investigates whether formal methods can serve as a viable alternative for targeted timing repair. We utilize the Z3 Optimization Modulo Theories (OMT) solver to model and optimize the worst-case register-to-register path in a design compiled with OpenROAD-flow-scripts (ORFS). Unlike iterative resizers that optimize one gate at a time, our solver considers the entire combinatorial space of the critical path simultaneously.

We evaluate this approach using the Greatest Common Denominator (GCD) design targeting the SkyWater 130nm High-Density (HD) library. Our contribution is a fully open-source, closed-loop pipeline that extracts physical data, formally optimizes it using Quantifier-Free Non-Linear Real Arithmetic (QF-NRA), and reapplies the solution to the layout to validate physical realizability.

II. BACKGROUND

Static Timing Analysis (STA) verifies the timing compliance of a digital circuit. The primary constraint of interest is the

Setup Time, which requires that data arrive at a capture flip-flop before the next clock edge. The slack for a path is defined as:

$$Slack_{setup} = T_{required} - T_{arrival} \quad (1)$$

where the required time depends on the clock period (T_{cycle}) and the library setup time (T_{setup}).

$$T_{arrival} \leq T_{cycle} - T_{setup} - T_{skew} \quad (2)$$

To fix violations (negative slack), EDA tools resize cells to higher drive strengths, which reduces delay but increases area and input capacitance.

III. METHODOLOGY

The project integrates two core components: the OpenROAD physical design toolchain and the Z3 OMT Solver. These components form an iterative pipeline: *extract* $\rightarrow$ *model* $\rightarrow$ *optimize* $\rightarrow$ *apply* $\rightarrow$ *validate*. A diagram of the core architecture is shown in Fig. 1.

A. Data Extraction via OpenROAD

We utilized ORFS to perform the complete RTL-to-GDSII flow. Once the design is routed, we developed a TCL extraction script, `extract_critical_path_reg.tcl`, which interfaces with the OpenSTA timing engine. It identifies the register-to-register path with the minimum slack (worst setup violation) and extracts detailed physical data, including wire resistance/capacitance (parasitics) and cell types for every stage.

B. Timing Variant Analysis

Formal optimization requires a continuous mathematical model of discrete library cells. We implemented a Python analyzer, `analyze_critical_path_reg.py`, to characterize the SkyWater 130nm library. The script parses Liberty (.lib) files and performs linear regression on the Non-Linear Delay Models (NLDM) to derive delay coefficients for every drive-strength variant of every cell on the path. This produces a linearized delay model:

$$Delay \approx a \cdot C_{load} + b \quad (3)$$

where a represents drive resistance and b represents intrinsic delay.

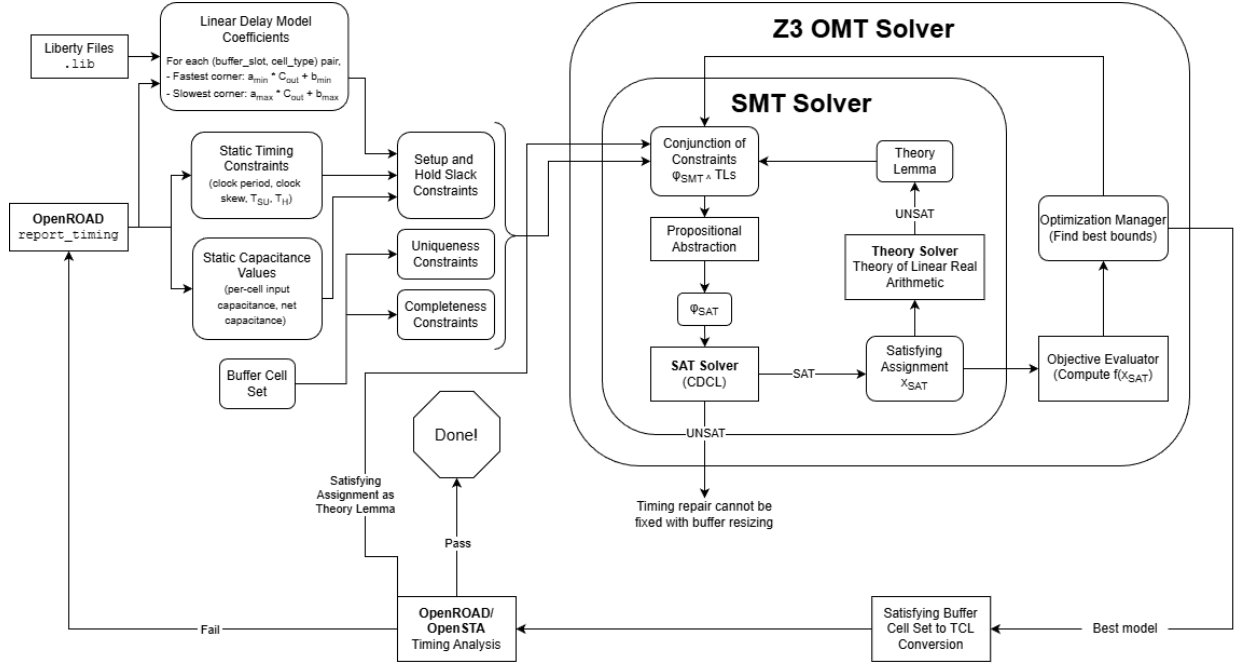


Fig. 1. Core Architecture: The closed-loop optimization pipeline. Physical data is extracted from OpenROAD (left), transformed into non-linear constraints, optimized by the Z3 OMT solver (right), and validated via OpenSTA.

C. OMT Formulation in Z3

The core of our approach is the translation of the physical path into a constraint satisfaction problem. We use Z3 in OMT mode with the Theory of Reals. The constraints are defined as follows:

1) *Structural Coherency*: For every resizeable slot s in the critical path, exactly one cell variant c must be selected from the library choices. We enforce this using a one-hot constraint on boolean decision variables $S_{s,c}$:

$$\sum_{c \in \text{Choices}(s)} \mathbb{I}(S_{s,c}) = 1, \quad \forall s \in \text{Slots} \quad (4)$$

where $\mathbb{I}$ is the indicator function mapping True/False to 1/0.

2) *Dynamic Capacitance Propagation*: A key innovation in our model is dynamic load propagation. The input capacitance C_{in} of a stage depends on the solver's choice for that stage:

$$C_{in}(s) = \sum_{c \in \text{Choices}(s)} \mathbb{I}(S_{s,c}) \cdot C_{lib_in}(c) \quad (5)$$

The output load C_{out} seen by a stage is the sum of the fixed wire capacitance and the dynamic input capacitance of the *next* stage:

$$C_{out}(s) = C_{wire}(s) + C_{in}(s_{next}) \quad (6)$$

3) *Non-Linear Delay Modeling*: The delay of each stage D_{stage} combines the cell delay and the interconnect delay (approximated via the Elmore model). Note that this creates non-linear constraints, as both the drive strength (a_c) and the load (C_{out}) are variable.

$$D_{cell}(s) = \sum_{c \in \text{Choices}(s)} \mathbb{I}(S_{s,c}) \cdot (a_c \cdot C_{out}(s) + b_c) \quad (7)$$

$$D_{net}(s) = R_{wire}(s) \cdot \left(\frac{C_{wire}(s)}{2} + C_{in}(s_{next}) \right) \quad (8)$$

4) *Timing Constraints*: The total path latency (AT) is the sum of the clock-to-Q delay and all stage delays. To be valid, this must fall within the clock period boundaries:

$$AT \leq T_{period} - T_{setup} - T_{skew} \quad (\text{Setup Check}) \quad (9)$$

$$AT \geq T_{hold} + T_{skew} \quad (\text{Hold Check}) \quad (10)$$

5) *Optimization Objectives*: We employ a lexicographic optimization strategy. The solver prioritizes objectives in a strict order:

- 1) **Maximize performance**: Maximize the setup slack to ensure timing closure.
- 2) **Minimize area**: Subject to the best possible timing found in step 1, minimize the total cell area.

$$\text{minimize} \left(\sum_{s \in \text{Slots}} \sum_{c \in \text{Choices}(s)} \mathbb{I}(S_{s,c}) \cdot \text{Area}(c) \right) \quad (11)$$

IV. RESULTS

We validated our flow on the GCD benchmark design with a target clock period of 4.0 ns.

A. Performance Comparison

After running the initial OpenROAD flow, the built-in resizer (`rsz`) successfully closed timing. We extracted the critical path and ran our OMT solver. The solver produced a configuration that improved the critical path slack by **0.02 ns** compared to the OpenROAD baseline.

While the magnitude of improvement is small, the result is significant. It demonstrates that formal methods can find global

optima that greedy heuristics miss. The solver was observed making counter-intuitive moves, such as downsizing upstream non-critical cells to reduce the capacitive load on previous stages; a global optimization strategy that local heuristics often fail to identify.

B. Solver Stability and Limitations

We observed that the solver’s performance was highly sensitive to the tightness of the constraints. At 4.0 ns, the solver converged in seconds. However, at tighter targets (e.g., 3.9 ns), the runtime increased dramatically. This is a known characteristic of Non-Linear Real Arithmetic (NRA); proving the non-existence of a solution near the boundary of the feasible region is computationally expensive.

V. CONCLUSION AND FUTURE WORK

This project demonstrates that SMT solvers can function effectively as synthesis engines, not just verification tools. By wrapping OpenROAD in a formal optimization loop, we established a methodology that prioritizes mathematical proofs of optimality over heuristics.

Future work to address scalability includes:

- **Linearization:** Pre-computing delay lookup tables to convert the problem from Non-Linear Arithmetic to Pseudo-Boolean (PB) optimization, significantly improving runtime.
- **Portfolio solvers:** Running multiple abstraction levels (SAT vs. OMT) in parallel.
- **Multi-Path optimization:** Extending the formulation to optimize disjoint paths simultaneously.

VI. WORK BREAKDOWN

Raphael Huang: Co-lead on Z3 implementation. Developed the initial solver boilerplate, constraint generation, CSV-to-JSON data pipelines, and the master Makefile orchestration.

Chad Baker: Co-lead on Z3 implementation. Designed the OMT configuration, implemented the physical constraints (capacitance/delay modeling), and coordinated data coherency between the OpenROAD extraction and the solver.

Wajid Ali: Co-lead on OpenROAD flow. Developed the TCL extraction scripts for OpenSTA and the Python parsers for analyzing Liberty NLDM tables and characterizing cell variants.

Anil Muddamsetti: Co-lead on OpenROAD flow. Managed the environment setup (Docker, OpenROAD build), evaluated design targets, and refined the final report structure and quality.

ACKNOWLEDGMENT

We would like to thank Professor Daniel Fremont for his guidance on Formal Methods and SMT solver applications.

APPENDIX

The complete source code, including installation instructions, Docker setup, and the Makefile workflow, is available at: <https://github.com/rphlhuang/ORFS-SMT-RepairTiming>

To replicate the results:

- 1) Install OpenROAD-flow-scripts
- 2) Clone the repository inside the `flow/` folder within the ORFS repo root.
- 3) Run `make setup` to initialize the environment.
- 4) Run `make` to execute the full closed-loop pipeline.